



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

计算机系统实验

第一次实验

CONTENTS

目录

「01」

实验总体安排

「02」

实验目的

「03」

实验任务

「04」

作业提交



本学期实验总体安排



➤ 实验总分：20分，总共8学时

实验项目	内容	提交	分数	课时
Lab1: Buflab	对包含缓冲区的目标程序进行溢出攻击，通过覆盖、修改栈中保存内容改变程序行为	代码及实验报告	10	4
Lab2: TinyShell	实现一个具备作业控制 (Job Control) 功能的TinyShell程序	代码及实验报告	10	4

➤ 课程主页及指导书（校内网）：<https://comp3052.p.cs-lab.top>

➤ 实验文件下载地址（校内网）：<https://file.cs-lab.top/s/BtJy7CRzHmmlpsQ>



实验背景与目标——Lab1 Buflab



➤ 实验背景

- 计算机系统安全基础
- 程序运行时内存模型（栈结构、函数调用约定）
- 缓冲区溢出漏洞的原理与危害

➤ 实验目标

- 理解C语言函数的汇编级实现及缓冲区溢出原理
- 通过构造恶意输入完成不同难度等级的攻击（Level 0~4）
- 掌握GDB调试与反汇编分析技术



实验数据与文件

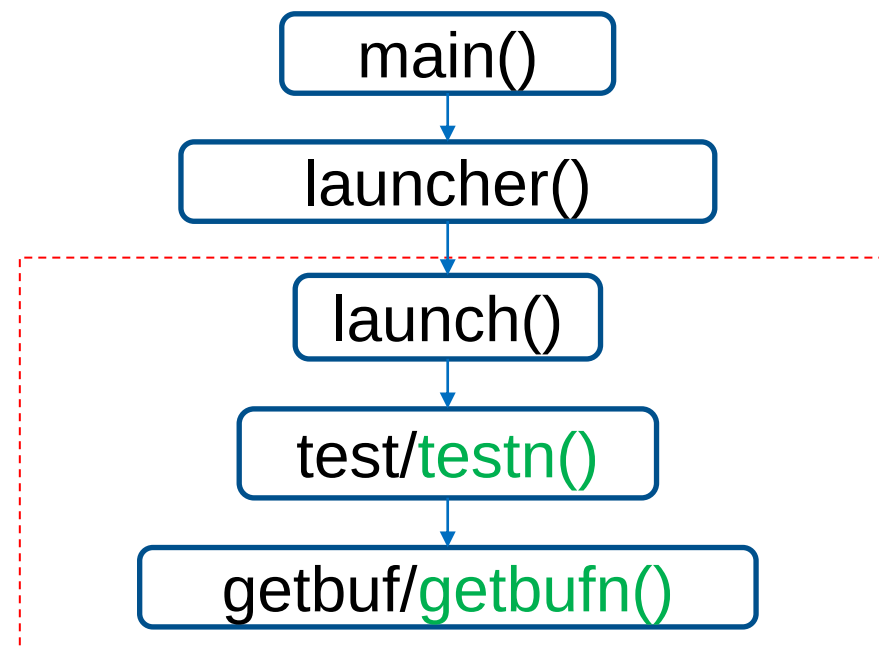


- **实验文件**下载地址（校内网）：<https://file.cs-lab.top/s/BtJy7CRzHmmlpsQ>
- **学生实验数据包**：<学号>.tar
- **实验环境**：Linux 64-bit（可以使用[Ubuntu虚拟机](#)，也可以利用自己搭建的 Linux 环境）
- **解压命令**：tar xf <学号>.tar （每位同学不一样）
- 数据包中主要包含下列文件：
 - **bufbomb**：可执行程序，攻击目标程序
 - **makecookie**：由学号产生4字节序列，如0x5f405c9a，称为“cookie”。用作实验中需要置入栈中的数据之一并区分不同学生的实验。
 - **hex2raw**：可执行程序，字符串格式转换程序



➤ bufbomb命令行参数:

- **-u userid**: bufbomb程序将基于userid (学号) 决定内部使用的cookie值 (同makecookie程序的输出), 且在bufbomb程序内部, 一些关键的栈地址取决于userid所对应的cookie值。
- **-h**: 打印可用命令行参数列表
- **-n**: 以 “Nitro”模式运行, 专用于Level 4实验阶段。



- ◆ main函数里launcher函数被调用cnt次, 但除了最后Nitro阶段, cnt都只是1。
- ◆ testn、getbufn仅在Nitro阶段 (Level 4) 被调用, 其余阶段均调用test、getbuf。



Bufbomb实验分析



- **缓冲区溢出点：函数Gets()不判断buf大小，字符串超长，缓冲区溢出**
- **正常情况：** 在getbuf执行完其返回语句，程序从test函数的第7行开始继续执行。
- **攻击原理：**
 - 覆盖返回地址 → 跳转至注入代码
 - 利用NOP雪橇增加命中概率

```
int getbuf() {  
    char buf[NORMAL_BUFFER_SIZE];  
    // NORMAL_BUFFER_SIZE是大于等于32的一个常数  
    gets(buf);  
    //从标准输入流输入字符串，gets存在缓冲区溢出漏洞  
    return 1;  
    //当输入字符串超过32字节即可破坏栈帧结构  
}
```

```
void test() {  
    int val;  
    /* Put canary on stack to detect possible corruption */  
    volatile int local = uniqueval();  
    val = getbuf();  
    /* Check for corrupted stack */  
    if (local != uniqueval()) {  
        printf("Sabotaged!: the stack has been corrupted\n");  
    }  
    else if (val == cookie) {  
        printf("Boom!: getbuf returned 0x%x\n", val);  
        validate(3);  
    } else {  
        printf("Dud: getbuf returned 0x%x\n", val);  
    }  
}
```

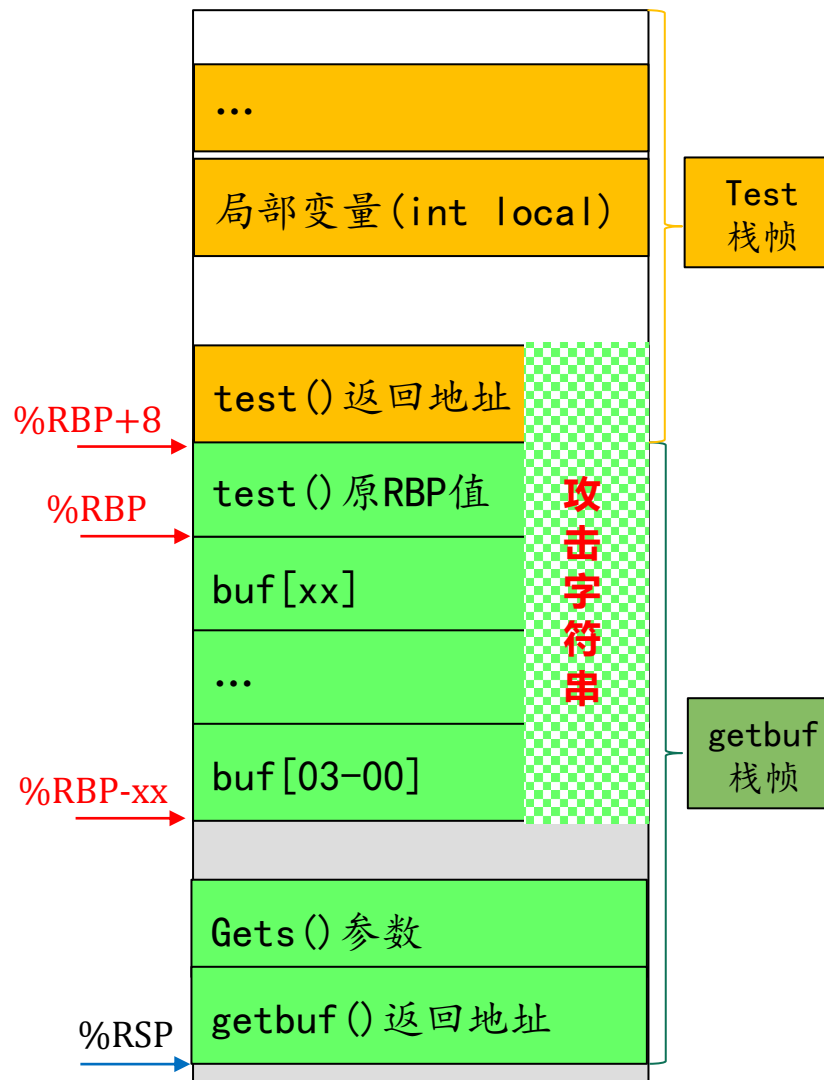


攻击手段



- 设计字符串输入给bufbomb，造成缓冲区溢出，使bufbomb程序完成一些有趣的事情
- **攻击字符串：**
 - 无符号字节数据，十六进制表示，字节间用空格隔开
 - 如：68 ef cd ab 00 83 c0
 - 与cookie相关，每位同学的攻击字串不同
 - 为输入方便将攻击字符串写在文本文件中

执行getbuf函数时的栈帧





- 如果用户输入给getbuf()的字符串不超过(NORMAL_BUFFER_SIZE-1)个字符长度的话，很明显getbuf()将正常返回1，如下列运行示例所示：

```
linux> ./bufbomb -u 123456789
```

```
Type string: I love ICS.
```

```
Dud: getbuf returned 0x1
```

- 如果输入一个更长的字符串，通常会发生类似下列的错误：

```
linux> ./bufbomb -u 123456789
```

```
Type string: I still maintain the point that designing a monolithic kernel  
in 1991 is a fundamental error. Be thankful you are not my student. You  
would not get a high grade for such a design.
```

```
Ouch!: You caused a segmentation fault!
```

- 正如上面的错误信息所示，缓冲区溢出通常导致程序状态被破坏，产生存储器访问错误。
(为什么会产生一个段错误？x86的栈结构如何组成？)



实验任务



对一个可执行程序“bufbomb”实施一系列5个难度递增的缓冲区溢出攻击缓冲区溢出攻击 (buffer overflow attacks)

➤ 5个难度级逐级递增，分别命名为：

Level 0: smoke (让目标程序调用smoke函数)

Level 1: fizz (让目标程序使用特定参数调用Fizz函数)

Level 2: bang (让目标程序调用Bang函数并修改全局变量)

Level 3: boom (无感攻击，并传递有效返回值)

Level 4: kaboom (栈帧地址变化时的有效攻击)

■ 需要调用的函数均在目标程序中存在（起始指令地址可知）

每级需根据任务设计构造1个攻击字符串，对目标程序实施缓冲区溢出攻击



任务1: Smoke



构造攻击字符串作为目标程序输入，造成缓冲区溢出，使getbuf()返回时不返回到test函数，而是转向执行smoke

```
void smoke()  
{  
    printf("Smoke!: You called smoke() \n");  
    validate(0);  
    exit(0);  
}
```

- 用来推断攻击字符串的所有信息可从bufbomb反汇编代码中获得
 - objdump -d
- 可使用GDB工具单步跟踪getbuf函数的最后几条指令，以了解程序的运行情况

```
linux>cat smoke.txt | ./hex2raw | ./bufbomb -u 123456789  
Userid: 123456789  
Cookie: 0x25e1304b  
Type string:Smoke!: You called smoke()  
VALID  
NICE JOB!
```

攻击成功界面



任务1: Smoke



攻击字符串的大小:

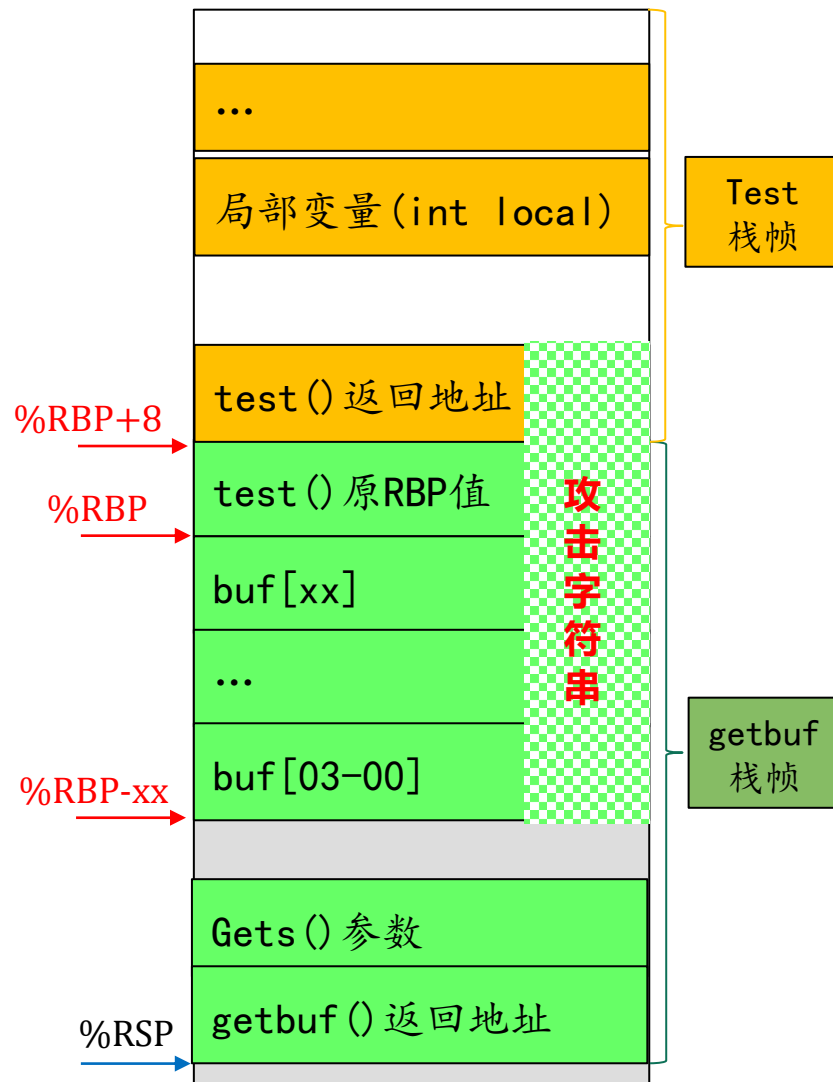
buf大小 +
8 (test()原RBP值) +
8 (test()返回地址)

000000000401c11 <getbuf>:

```
401c11: 55          push  %rbp
401c12: 48 89 e5    mov   %rsp,%rbp
401c15: 48 83 ec 30 sub   $0x30,%rsp
401c19: 48 8d 45 d0 lea    -0x30(%rbp),%rax
401c1d: 48 89 c7    mov   %rax,%rdi
401c20: e8 37 fa ff ff call  40165c <Gets>
401c25: b8 01 00 00 00 mov   $0x1,%eax
401c2a: c9          leave
401c2b: c3          ret
```

一个getbuf反汇编示例

执行getbuf函数时的栈帧





任务2: Fizz



构造攻击字符串造成缓冲区溢出，使目标程序调用fizz函数，并将cookie值作为参数传递给fizz函数，使fizz函数中的判断成功，需仔细考虑将cookie放置在栈中什么位置

■ 程序无需真的调用fizz（只需执行fizz函数的语句代码）

- 攻击（修改）返回地址区域
- 修改被引用的栈存储数值
 - 地址-0x4(%rbp)和0x3e3f(%rip)指向的存储器内容相同
 - 两个地址-0x4(%rbp)和0x3e3f(%rip)相等

0000000004013e5 <fizz>:

4013e5: 55	push %rbp
4013e6: 48 89 e5	mov %rsp,%rbp
4013e9: 48 83 ec 10	sub \$0x10,%rsp
4013ed: 89 7d fc	mov %edi,-0x4(%rbp)
4013f0: 8b 55 fc	mov -0x4(%rbp),%edx
4013f3: 8b 05 3f 3e 00 00	mov 0x3e3f(%rip),%eax
4013f9: 39 c2	cmp %eax,%edx
4013fb: 75 20	jne 40141d <fizz+0x38>

当前指令地址

RIP相对寻址示意图

↓
0x4013f3: 8b 05 3f 3e 00 00 ← 6字节指令

操作码	偏移量 (0x3e3f)

下一条指令地址 = 0x4013f3 + 6 = 0x4013f9
目标地址 = 0x4013f9 + 0x3e3f = 0x405238

```
void fizz(int val){
    if (val == cookie) {
        printf("Fizz!: You called fizz(0x%x)\n", val);
        validate(1);
    } else
        printf("Misfire: You called fizz(0x%x)\n", val);
    exit(0);
}
```



任务3: bang



构造攻击字符串，使目标程序调用bang函数，要将函数中全局变量global_value篡改为cookie值，使相应判断成功，需要在缓冲区中注入恶意代码篡改全局变量。

```
int global_value = 0;
void bang(int val){
    if (global_value == cookie) {
        printf("Bang!: You set global_value to 0x%x\n",
global_value);
        validate(2);
    }
    else
        printf("Misfire: global_value = 0x%x\n", global_value);
    exit(0);
}
```

➤提示：攻击代码应首先将全局变量global_value设置为对应userid的cookie值，再将bang函数的地址压入栈中，然后执行一条ret指令从而跳至bang函数的代码执行



任务3: bang



调用其他函数

- 攻击返回地址区域

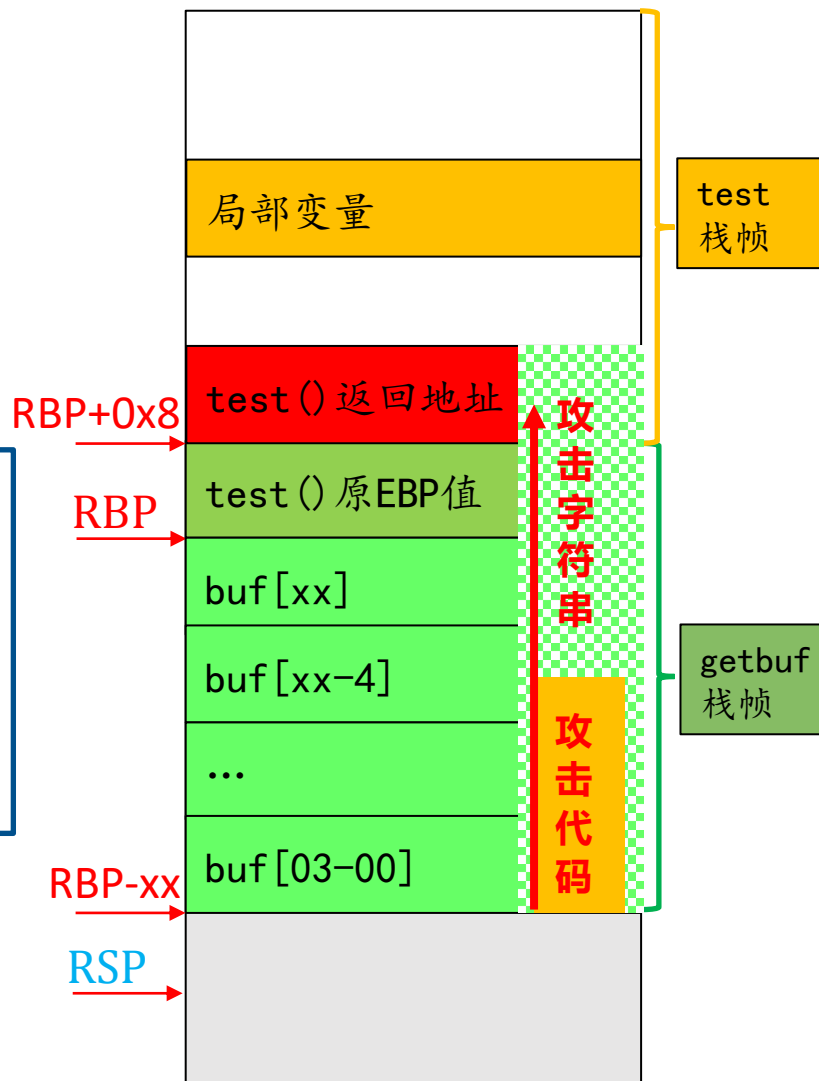
篡改全局变量

- 简单字符串覆盖做不到
- 需编写恶意代码，插入到攻击字符串合适位置

攻击代码

- 将全局变量`global_value`设置为对应userid的`cookie`值 (`mov指令`)
- 再将bang函数的地址压入栈中 (`push指令`)
- 跳至bang函数的代码执行 (`ret指令`)

- 当被调用函数返回时，应先转向这段恶意代码
- 恶意代码负责篡改全局变量，并跳转到bang函数





任务3: bang



- 可以使用GDB获得构造攻击字符串所需的信息。例如，在getbuf函数里设置一个断点并执行到该断点处，进而确定global_value和buf缓冲区等变量的地址
- 如何构造含有恶意攻击代码的攻击字符串？
 - 编写汇编代码文件asm.s，将该文件编译成机器代码
 - gcc -m32 -c asm.s
 - 反汇编asm.o得到恶意代码字节序列，插入攻击字符串适当位置
 - objdump -d asm.o

```
linux>cat bang.txt | ./hex2raw | ./bufbomb -u 123456789
Userid: 123456789
Cookie: 0x25e1304b
Type string:Bang!: You set global_value to 0x25e1304b
VALID
NICE JOB!
```

攻击成功界面



任务4: boom



- 前3次攻击都是使目标程序**跳转到特定函数**，进而利用exit函数结束目标程序运行，攻击造成的**栈帧结构破坏**是可接受的。
- Boom要求更高明的攻击，**要求被攻击程序能返回到原调用函数test继续执行**——即调用函数感觉不到攻击行为。
- **挑战**
 - 还原对栈帧结构的任何破坏



任务4: boom



- 构造攻击字符串，使得getbuf都能将正确的cookie值返回给test函数，而不是返回值1。

- **攻击成功界面**

```
linux>cat boom.txt | ./hex2raw | ./bufbomb -u 123456789
Userid: 123456789
Cookie: 0x25e1304b
Type string:Boom!: getbuf returned 0x25e1304b
VALID
NICE JOB!
```

注：这里，boom不是一个函数

```
int getbuf() {
    char buf[NORMAL_BUFFER_SIZE];
    // NORMAL_BUFFER_SIZE是大于等于32的一个常数
    gets(buf);
    //从标准输入流输入字符串，gets存在缓冲区溢出漏洞
    return 1;
    //当输入字符串超过32字节即可破坏栈帧结构
}
```



任务4: boom



➤ 将cookie值返回给test函数

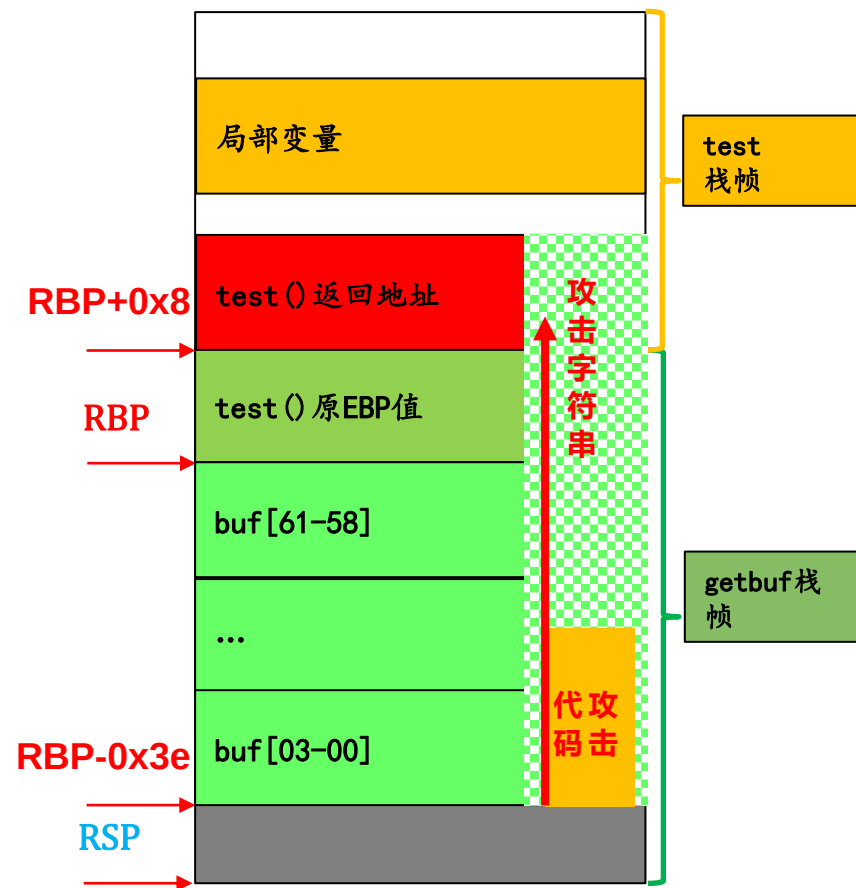
- 可通过%RAX寄存器传递

➤ 还原任何被破坏的栈帧状态

- getbuf栈帧中保存的、需要恢复的test运行现场状态数据 (%RBP值, 可用GDB获取)

➤ 返回到原来的调用函数test执行

- 使用push指令将getbuf()后的第一条指令地址压入栈中
- 执行ret指令从该地址开始继续执行test()函数





任务5: kaboom



- **背景：**一个给定函数的栈帧的确切内存地址通常随程序运行实例的不同而不同
 - 之前级别实验中getbuf具有固定的栈帧地址（不随不同运行实例而不变）
- 本实验级别在调用testn函数前会在栈上分配一随机大小的内存块，因此testn函数及其所调用的getbufn函数的栈帧起始地址在每次运行程序时具有一个随机的、不固定的值
- **注意：本级别运行bufbomb程序时需如下指定“-n”命令行开关（Nitro模式）：**
linux> cat kaboom.txt | ./hex2raw -n | ./bufbomb -n -u 123456789
 - bufbomb使用输入攻击字符串连续执行5次getbufn函数（每次采用不同的栈偏移位置），程序每次均能返回cookie值才算成功

```
linux>$ cat kaboom.txt | ./hex2raw -n | ./bufbomb -n -u 123456789
Userid: 123456789
Cookie: 0x25e1304b
Type string:KABOOM!: getbufn returned 0x25e1304b
Keep going
Type string:KABOOM!: getbufn returned 0x25e1304b
Keep going
Type string:KABOOM!: getbufn returned 0x25e1304b
Keep going
Type string:KABOOM!: getbufn returned 0x25e1304b
Keep going
Type string:KABOOM!: getbufn returned 0x25e1304b
VALID
NICE JOB!
```

攻击成功界面



任务5: kaboom

- **任务：构造攻击字符串，使得输入其至bufbomb目标程序后**
 - **getbufn函数返回cookie值至testn函数，而不是返回值1**
 - **复原/清除所有被破坏的状态，将正确的返回位置压入栈中，并执行ret指令以返回testn函数**

getbufn函数中缓冲区扩大到512字节以上，以方便构造更可靠的攻击代码

```
void testn(){
    int val;
    volatile int local = uniqueval();
    val = getbufn();

    /* Check for corrupted stack */
    if (local != uniqueval()) {
        printf("Sabotaged!: the stack has been corrupted\n");
    }
    else if (val == cookie) {
        printf("KABOOM!: getbufn returned 0x%x\n", val);
        validate(4);
    }
    else {
        printf("Dud: getbufn returned 0x%x\n", val);
    }
}
```

```
int getbufn(){
    char buf[KABOOM_BUFFER_SIZE];
    Gets(buf);
    return 1;
}
```



任务5: kaboom



➤ getbufn()返回时转去执行攻击字符串中指令

- 用以覆盖返回地址的指令起始地址（原等于buf首地址）**不固定**

- buf前部填充**nop指令**（机器代码0x90）——nop雪橇
- 指令起始地址 = GDB获得大致buf首地址 + 0.5*buf大小
指向nop雪橇的中间位置（最大程度容纳stack上下偏移）

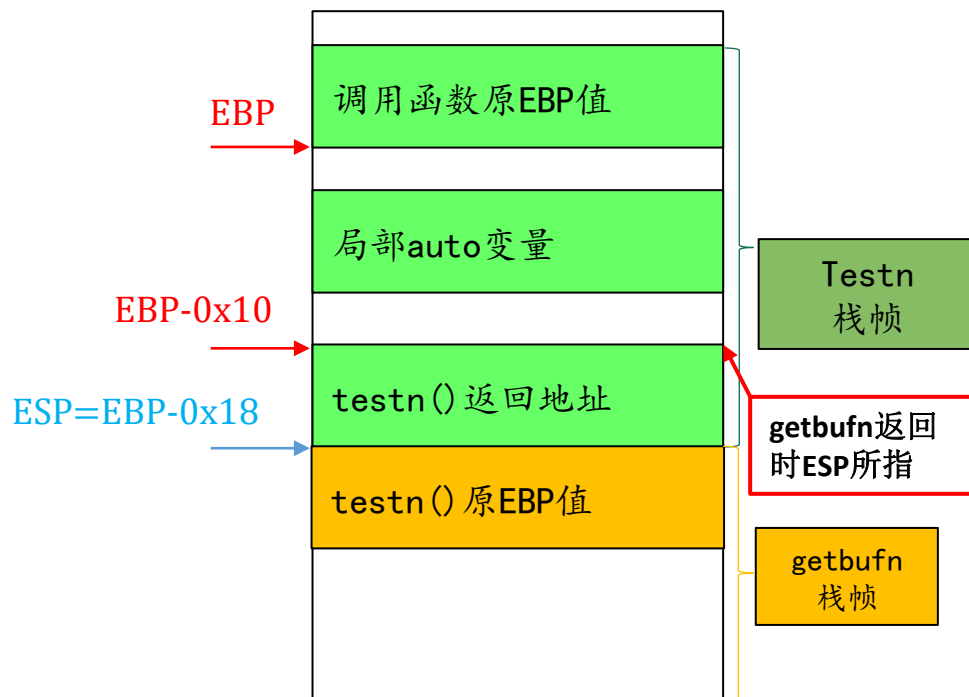
➤ 还原任何被破坏的栈帧状态

- testn函数栈帧的原EBP值**不固定**

- 可从调用getbufn后的%ESP（未被栈中的攻击字符串内容所覆盖）间接获得（从testn的汇编代码推导出），进而用movl指令直接设置%EBP

00000000040151f <testn>:

40151f: 55	push %rbp
401520: 48 89 e5	mov %rsp,%rbp
401523: 48 83 ec 10	sub \$0x10,%rsp
401527: b8 00 00 00 00	mov \$0x0,%eax
40152c: e8 84 04 00 00	call 4019b5 <uniqueval>
401531: 89 45 f8	mov %eax,-0x8(%rbp)
401534: b8 00 00 00 00	mov \$0x0,%eax





- ① **objdump**: 反汇编bufbomb可执行目标文件。然后查看实验中需要的大量地址、栈帧结构等信息。
- ② **gdb**: 目标程序没有调试信息，无法通过单步跟踪观察程序的执行情况。但依然需要设置断点让程序暂停，并进而观察必要的内存、寄存器内容等，尤其对于阶段2~4，观察寄存器，特别是ebp的内容是非常重要的。
 - 断点设置: `b *getbuf`
 - 查看栈帧: `x/32xw $esp`
 - 跟踪执行: `stepi / nexti`



- **gcc**: 在阶段3~5, 你需要编写少量的汇编代码, 然后用gcc编译成机器指令, 再用objdump反汇编成机器码, 以此来构造包含攻击代码的攻击字符串。
- **返回地址**: test函数调用getbuf后的返回地址是getbuf后的下一条指令的地址 (通过观察bufbomb反汇编代码可得)。而带有攻击代码的攻击字符串所包含的攻击代码地址, 则需要你在深入理解地址概念的基础上, 找到它们所在的位置并正确使用它们实现程序控制的转向。



攻击字符串文件的提交要求



➤ 提交对应Level 0 - 4的最多5个solution文件

- 文件命名方式为 “级别.txt”

- 如： **smoke.txt**、 **fizz.txt**、 **bang.txt**、 **boom.txt**、 **kaboom.txt**

- 攻击字符串的代码序列格式：

- 两个16进制值作为一个16进制对，每个16进制对代表一个字节，每个16进制对之间用空格分开。例如 “68 ef cd ab 00 83 c0 11 98 ba dc fe”

➤ 用tar命令打包为 “学号.tar”文件提交

Linux > tar cf 123456789.tar smoke.txt fizz.txt ...

注意：TAR文件中一定不要包含任何目录结构——即在上述命令行中指定待打包的源文件时，不要包含目录前缀（因此在运行tar命令前应cd到源文件所在目录或者使用 “-C” 命令选项）



提交内容：实验报告（有模板） + <学号>.tar

截止时间：

实验课后两周内提交至HITsz Grader 作业提交平台，具体截止日期参考平台发布。

- 登录网址：： <http://grader.tery.top:8000/#/login>
- 推荐浏览器： Chrome
- 初始用户名、密码均为学号，登录后请修改

注意

上传后可自行下载以确认是否正确提交



**同学们
请开始实验吧！**